

CURSO TECNICO · DESENVOLVIMENTO WEB

Guia do Aluno

Como usar, entender e ampliar o Framework MVC Didatico em PHP, do primeiro comando ate um CRUD com autenticacao, relacoes, testes e relatorios PDF.

EDICAO ATUALIZADA · AGOSTO DE
2026

Framework MVC - Curso Tecnico

PHP 8.1 ou superior · SQLite ou MySQL/MariaDB

Sem Composer e sem dependencias externas

Como usar este guia

O projeto é um framework pequeno para estudar a separação entre modelo, controlador e view. Ele começa vazio: você cria as tabelas e os recursos com comandos do próprio projeto.

Regra desta edição. O instalador prepara o banco, mas não cria tabelas da aplicação nem dados de exemplo. O comando `scaffold:crud` é o ponto de partida para cada recurso.

1. Preparar e executar o projeto	3
2. Entender o MVC e o caminho da requisição	4
3. Gerar um CRUD com o scaffold	6
4. Criar uma relação belongs_to	8
5. Adicionar autenticação	9
6. Gerar relatórios PDF	11
7. Personalizar modelos, controllers e views	12
8. Escrever e executar testes	14
9. Configuração e problemas comuns	15

A referência curta de comandos está em `documentacao/Referencia-Comandos.md`. Este PDF acompanha o passo a passo de `documentacao/Tutorial-Comandos.md`.

1. Preparar e executar o projeto

Requisitos

- PHP 8.1 ou superior;
- SQLite ou MySQL/MariaDB;
- um terminal aberto na pasta do projeto.

O framework nao usa Composer. O autoloader proprio carrega as classes a partir dos namespaces e das pastas do projeto.

Conferir a instalacao

```
php testes/executar.php
```

O resultado esperado termina com `TUDO CERTO! O sistema esta funcionando.` . Se estiver usando MySQL, inicie o servidor antes do instalador e confira `configuracoes/banco.php` .

Preparar o banco

```
php instalar.php
```

O instalador cria o banco MySQL quando necessario, executa o esquema do driver e deixa o banco pronto. A mensagem informa que nenhuma tabela padrao foi criada.

Importante. O instalador nao recria o banco do zero e nao apaga os registros do projeto. Depois de criar um recurso pelo console, rode o instalador apenas quando quiser preparar o banco em outra maquina ou ambiente.

Rodar o servidor

```
php -S localhost:8000 roteador.php
```

Abra `http://localhost:8000` . O arquivo `roteador.php` encaminha as URLs para `index.php` quando o servidor embutido do PHP e usado.

2. Entender o MVC e o caminho da requisicao

Model

Fica em `modelos/` . Conversa com o banco, oferece o CRUD por heranca e concentra validacoes e consultas especificas.

View

Fica em `views/` . Monta o HTML com os dados recebidos. Dados exibidos devem passar por `e()` .

Controller fica em `controllers/` . Recebe a requisicao, chama o modelo e escolhe a view. A regra de ouro e simples: SQL em modelos, HTML em views, coordenacao em controllers.

O caminho de uma URL

1. O servidor encaminha a requisicao para `index.php` .
2. `nucleo/bootstrap.php` carrega configuracoes, autoloader, helpers e sessao.
3. `Nucleo\App` divide a URL em controlador, metodo e parametros.
4. O controller e instanciado e seu metodo publico e executado.
5. O controller chama o modelo e renderiza uma view dentro de `views/template/layout.php` , ou envia JSON/PDF.

URL	Classe e metodo
/	HomeController::index()
/produtos	ProdutosController::index()

/produtos/ver/1	ProdutosController::ver('1')
/produtos/relatorio	ProdutosController::relatorio()

Somente metodos publicos viram rotas. Metodos `private` e `protected` continuam disponiveis para apoio interno do controller.

3. Gerar um CRUD com o scaffold

Informe o nome da tabela e pelo menos um campo no formato `campo:tipo`:

```
php console.php scaffold:crud produtos nome:string preco:decimal estoque:integer
```

O comando valida os nomes, escreve os arquivos, adiciona a tabela aos dois esquemas e executa a migracao do driver configurado. Arquivos que ja existem nao sao sobrescritos.

Arquivos gerados

Arquivo	Responsabilidade
<code>modelos/Produto.php</code>	Classe do modelo, tabela e campos preenchiveis.
<code>controllers/ProdutosController.php</code>	Acoes CRUD e acao de relatorio.
<code>views/produtos/</code>	Listagem, formulario e detalhe.
<code>testes/modelos/ProdutoTest.php</code>	Teste automatico do CRUD do modelo.
<code>testes/controllers/ProdutosControllerTest.php</code>	Teste das rotas e do relatorio protegido.
<code>banco/esquema.sqlite.sql</code> e <code>esquema.mysql.sql</code>	Estrutura para os dois drivers.

Tipos aceitos

Comando	SQLite	MySQL	Campo HTML
<code>string</code>	TEXT	VARCHAR(255)	texto
<code>text</code>	TEXT	VARCHAR(255)	texto
<code>integer</code>	INTEGER	INT	numero
<code>decimal</code>	REAL	DECIMAL(12,2)	numero
<code>boolean</code>	INTEGER	TINYINT(1)	checkbox
<code>date</code> , <code>datetime</code> , <code>time</code>	TEXT	DATE, DATETIME, TIME	data/hora

`id` e criado automaticamente. `criado_em` e reservado. Os nomes precisam comecar por letra minuscula e aceitar somente letras, numeros e sublinhado.

Rotas geradas

Rota	Funcao
<code>/produtos</code>	Lista registros.
<code>/produtos/criar</code>	Mostra o formulario.
<code>/produtos/salvar</code>	Grava via POST.
<code>/produtos/ver/1</code>	Mostra um registro.
<code>/produtos/editar/1</code>	Abre a edicao.
<code>/produtos/atualizar/1</code>	Atualiza via POST.
<code>/produtos/excluir/1</code>	Exclui um registro.
<code>/produtos/relatorio</code>	Gera PDF filtravel e protegido.

4. Criar uma relacao belongs_to

Crie primeiro a tabela pai. Depois crie a tabela filha usando `campo_id:belongs_to=tabela_pai`:

```
php console.php scaffold:crud turmas nome:string
php console.php scaffold:crud matriculas nome:string turma_id:belongs_to=turmas
```

O scaffold transforma `turma_id` em inteiro, cria a chave estrangeira nos esquemas, gera no modelo o metodo `turmas()` e envia as opcoes para o select do formulario.

A ordem importa. A tabela apontada deve existir antes da tabela filha. Se a relacao for adicionada manualmente, mantenha a mesma definicao em SQLite e MySQL.

5. Adicionar autenticacao

Aplicar a um modelo existente

```
php console.php scaffold:crud clientes nome:string
php console.php auth:install Cliente
```

O comando adiciona `email` e `senha` se faltarem, atualiza o modelo com `Nucleo\Autenticavel` e cria as telas e o controller de autenticacao.

Rotas de autenticacao

Rota	Funcao
<code>/auth/login</code>	Entrar.
<code>/auth/registrar</code>	Criar conta.
<code>/auth/sair</code>	Encerrar sessao.

Credenciais obrigatorias. E-mail e senha sao obrigatorios, e a senha precisa ter pelo menos 6 caracteres. O model e o formulario aplicam essa regra. Em uma tabela vazia, as colunas recebem `NOT NULL`.

As senhas sao gravadas com `password_hash()` e conferidas com `password_verify()`. O comando tambem pode criar o modelo padrao quando executado sem argumento:

```
php console.php auth:install
```

Varios providers (opcional)

Um projeto pode ter mais de uma tabela autenticavel, mas isso nao e ativado automaticamente. O primeiro provider continua usando `/auth`. Para gerar outra tela de login, informe um prefixo exclusivo:

```
php console.php scaffold:crud professores nome:string
php console.php auth:install Professore professor
```

Esse comando cria um controller e telas proprios em `views/auth/professor/`, com as rotas `/auth-professor/login`, `/auth-professor/registrar` e `/auth-professor/sair`. A sessao tambem e separada.

```
->exigirAutenticacao("professor");
autenticado("professor");
usuario_id("professor");
```

O provider padrao continua funcionando sem prefixo. O gerador nao sobrescreve arquivos existentes; escolha outro prefixo se necessario.

Proteger uma acao

```
public function index(): void
{
    $this->exigirAutenticacao();
    $this->view('relatorios/index', ['titulo' => 'Relatorios']);
}
```

O scaffold ja coloca essa verificacao na rota `{tabela}/relatorio`. As demais rotas do CRUD nao sao protegidas automaticamente;

adicione a chamada nas acoes que exigirem usuario.

Use `autenticado()` para saber se existe sessao e `usuario_id()` para obter o id do usuario atual.

6. Gerar relatorios PDF

Relatorio pela web

A listagem gerada inclui o link **Relatorio PDF**. A acao passa pelo controller, exige autenticao e retorna o PDF em memoria:

```
/produtos/relatorio
/produtos/relatorio?nome=teclado&estoque=1
```

Qualquer campo do recurso pode ser usado como filtro parcial; `id` tambem e aceito. Os valores sao enviados como parametros PDO, e nao concatenados ao SQL.

Por que nao linkar um arquivo? Arquivos estaticos sao publicos. A rota web garante que a sessao seja verificada antes da consulta e da resposta do PDF. O link simples da listagem abre o relatorio sem filtros; para filtrar, use uma URL GET ou um formulario GET.

Exportacao offline

```
php console.php relatorio:pdf produtos
php console.php relatorio:pdf Produto relatorios/produtos.pdf
```

O comando aceita tabela ou nome do modelo. Sem o segundo argumento, grava em `relatorios/{tabela}.pdf`. O gerador e nativo, pagina os registros e nao precisa de Composer.

O arquivo offline e apropriado para uma exportacao executada no terminal. Nao aponte uma pagina protegida diretamente para um PDF dentro de uma pasta publica.

7. Personalizar modelos, controllers e views

O modelo gerado

```
class Produto extends \Nucleo\Model
{
    protected string $tabela = 'produtos';
    protected array $preenchiveis = ['nome', 'preco', 'estoque'];
    protected string $ordemPadrao = 'id DESC';
}
```

O modelo ja oferece `todos()`, `buscar()`, `contar()`, `criar()`, `atualizar()`, `excluir()`, `onde()` e `primeiroOnde()`. Para consultas proprias, use `consultar($sql, $parametros)`.

```
public function porNome(string $nome): array
{
    return $this->consultar(
        'SELECT * FROM produtos WHERE nome LIKE ? ORDER BY nome',
        [\Nucleo\Sql::comoLike($nome)]
    );
}
```

Seguranca SQL. Valores sempre vao nos parametros `?`. Nomes de tabela, coluna e ordenacao passam pelas validacoes de `Nucleo\Sql`. Nunca concatene texto recebido do usuario no SQL.

Validacao

```
public function validar(array $dados, int|string|null $ignorarId = null): array
{
    $v = new \Nucleo\Validador($dados);

    $v->obrigatorio('nome', 'Nome')
        ->minimo('nome', 3, 'Nome');
```



```
return $v->erros();
}
```

Views e layout

As views ficam em `views/<recurso>/`. O template comum é `views/template/layout.php`. Escape o que vem do banco ou do usuário:

```
<td><? e($registro['nome'] ?? '') ?></td>
<a href="<? url('produtos/editar/' . $registro['id']) ?>">Editar</a>
```

Use `url()` para rotas internas e `asset()` para CSS, JavaScript e imagens. Assim o sistema funciona tanto na raiz do servidor quanto em uma subpasta.

8. Escrever e executar testes

Comandos

```
php testes/executar.php
php testes/executar.php ProdutoTest
php testes/executar.php Controllers
```

Os testes usam SQLite em memória e não alteram o banco MySQL ou o arquivo SQLite da aplicação.

O que o scaffold testa

- o teste de modelo verifica criar, buscar, contar, atualizar e excluir;
- o teste de controller verifica listagem, formulário, cadastro, detalhe, edição e exclusão;
- o teste de controller também verifica o link `/<{tabela}>/relatorio`, o redirecionamento para login sem sessão e o PDF autenticado com filtro GET.

Um teste novo herda de `Testes\Suporte\TesteBase`. Métodos públicos iniciados por `teste` são executados automaticamente.

Ajuda	Uso
<code>requisitar('produtos')</code>	Simula GET.
<code>postar('produtos/salvar', \$dados)</code>	Simula POST.
<code>limparTabela('produtos')</code>	Limpa a tabela no preparar.
<code>limparSessao()</code>	Remove sessão e mensagens do cenário.
<code>assertIgual()</code> , <code>assertContem()</code>	Comparam valores e textos.

9. Configuração e problemas comuns

Arquivos principais

Arquivo	O que ajustar
<code>configuracoes/app.php</code>	Nome, URL base, debug, fuso e rota inicial.
<code>configuracoes/banco.php</code>	Driver, arquivo SQLite ou conexão MySQL.
<code>views/css/estilo.css</code>	Cores e layout.
<code>views/template/cabecalho.php</code>	Logo e menu.
<code>banco/esquema.sqlite.sql</code> e <code>esquema.mysql.sql</code>	Estrutura dos dois drivers.

Para usar SQLite, altere `'driver' => 'sqlite'` e rode `php instalar.php`. Para MySQL/MariaDB, inicie o servidor e confira host, porta, banco, usuário e senha.

Checklist rapido

Sintoma	Verificacao
Classe ou controller nao encontrado	Nome do arquivo, namespace e maiusculas precisam seguir a convencao.
View nao encontrada	<code>view('produtos/index')</code> aponta para <code>views/produtos/index.php</code> .
Campo nao salva	Confira o <code>name</code> do input, a coluna e <code>\$preenchiveis</code> .
Unknown column	Atualize os dois esquemas ou gere a coluna com o scaffold e rode a migracao.
Relatorio redireciona para login	Esse e o comportamento esperado sem <code>autenticacao_id</code> ou <code>usuario_id</code> na sessao.
Erro de banco	Confira o driver e a conexao em <code>configuracoes/banco.php</code> .
Pagina em branco	Durante o desenvolvimento, mantenha <code>'debug' => true</code> para ver a mensagem completa.

Fluxo recomendado

```
php instalar.php
php console.php scaffold:crud clientes nome:string email:string telefone:string
php console.php auth:install Cliente
php testes/executar.php
php -S localhost:8000 roteador.php
```

Crie a primeira conta em `/auth/registrar`, entre em `/auth/login` e use `/clientes`. Para exportar, abra `/clientes/relatorio` ou execute o comando offline no terminal.

Antes de publicar. Revise validacoes, proteja as rotas que exigem usuario e altere `app.debug` para `false`.